

HabotConnect Hiring Project

Junior Cloud & DevOps Engineer (GCP / Django / React) — Answer Document

Full Name:	Fathima Fiza C P
Contact Email:	Fathimafiza1612@gmail.com
Phone Number:	+91 8714261512
Date Submitted:	13 Sep. 2026
Repository Link:	https://github.com/fizaaaaaaa/habot-devops-task

1. Scenario and Objective

A junior developer pushed unencrypted API credentials into raw application code and triggered a database schema mismatch that broke downstream analytics. This project's objective was not to patch that single incident, but to build automated, mistake-proof ("Poka-Yoke") systems that make this category of error structurally difficult to ship in the first place.

Three tasks were completed to restore system integrity: secure Infrastructure-as-Code provisioning (Task 1), an automated fail-closed CI/CD build gate (Task 2), and strict schema and data validation (Task 3).

2. Task 1 — Terraform Secure Staging Provisioning

Objective

Provision a Google Cloud Storage "D0 Raw Landing" bucket and a BigQuery "D1 Staged/Enforced" dataset using Terraform, with strict IAM conditions and Row-Level Security policies.

Engineering Logic

- Encryption at rest: a dedicated Cloud KMS key (Customer-Managed Encryption Key) encrypts the raw landing bucket, rather than relying on Google's default encryption.
- Least privilege: the ingestion service account is granted only the `storage.objectCreator` role — it can write new files but cannot list, read, or delete existing ones.
- No public exposure: `uniform_bucket_level_access` and `public_access_prevention = "enforced"` remove any path to accidental public access, even by misconfiguration.
- Row-Level Security: a BigQuery row access policy restricts Learning Support Assistant (LSA) staff so that a query against the `students` table only returns rows where they are the assigned LSA, using the predicate `assigned_lsa_email = SESSION_USER()`.
- Versioning is enabled on the bucket so that an accidental overwrite or delete of raw data can be recovered.

Verified Outputs

```
raw_landing_bucket_name = "habot-devops-task-508507-d0-raw-landing"
raw_landing_bucket_url   = "gs://habot-devops-task-508507-d0-raw-landing"
staged_dataset_id        = "d1_staged_enforced"
students_table_id        = "habot-devops-task-508507.d1_staged_enforced.students"
```

3. Task 2 — Poka-Yoke Automated CI/CD Build Gate

Objective

Design a GitHub Actions pipeline that automatically lints, formats, and secret-scans every commit, and acts as a strict "fail-closed" gate — halting and quarantining any build with formatting errors or hardcoded secrets.

Engineering Logic

- Lint & Format job: runs Black and Flake8 against the Python code, and terraform fmt -check plus terraform validate against the infrastructure code.
- Secret Scan job: runs Gitleaks with a custom rule set tuned to catch generic hardcoded API keys and embedded GCP service-account private keys — directly addressing the scenario's root cause.
- Quarantine job: fires only when a prior gate fails, printing an explicit error stating the commit is quarantined and blocked from deploying — making the fail-closed behavior visible and auditable, not silent.
- Deploy job: only runs if both the lint and secret-scan jobs succeeded. Deployment is reached through explicit success, never through the mere absence of a detected failure.

Demonstrated Behavior

The pipeline was verified in both states:

- Bad commit: a hardcoded secret was intentionally introduced. The Secret Scan job failed, and the Quarantine job correctly triggered with the message "This commit is QUARANTINED and is blocked from deploying."
- Clean commit: the secret was removed (via git revert) and the full pipeline — lint, secret scan, and deploy — passed successfully.

4. Task 3 — Schema Mapping and DCYN Validation

Objective

Deconstruct an incoming student-onboarding JSON payload into a binary Yes/No logic library (DCYN), and write a Django REST Framework serializer with exact field validation limits that eliminates human judgment from the validation step.

Engineering Logic

- DCYN library (dcyn.py): a single function, to_dcyn(), accepts a fixed, explicit set of truthy and falsy values ("yes", "y", "true", "1", True, 1 / "no", "n", "false", "0", False, 0) and returns exactly "YES" or "NO". Any value outside that set — blank, "maybe", partial values, None — is rejected with a clear error rather than guessed at.
- Exact field limits: student_id must match the strict pattern STU-YYYY-NNNNN; date_of_birth is validated to imply a real, currently-a-minor age (0–18 years).
- Atomic validation: dcyn_batch() runs every DCYN field in the payload as one step. If any field is ambiguous, the entire record is rejected — there is no partial acceptance of a bad record.
- Business rule enforcement: a record without guardian consent is rejected in code (guardian_consent_given == "NO"), removing reliance on a human reviewer to catch this.

Demonstrated Behavior

The serializer was verified against three scenarios using `demo_run.py`:

- Valid payload: all four DCYN fields correctly normalized to YES/NO.
- Ambiguous value (e.g. "maybe" in place of a yes/no answer): cleanly rejected with a DCYN validation error.
- Missing guardian consent: cleanly rejected by the business rule, even though the value itself was a valid "NO".

See attached/linked screenshots of all three `demo_run.py` outputs.

5. Folder Structure Delivered

```
habot-devops-task/
├── README.md
├── task1-terraform/
│   ├── main.tf
│   ├── variables.tf
│   ├── outputs.tf
│   └── schemas/students_schema.json
├── .github/workflows/build-gate.yml    (Task 2 pipeline)
├── task2-cicd/
│   ├── .gitleaks.toml
│   └── demo_bad_commit_example.py
└── task3-django/
    ├── dcyn.py
    ├── models.py
    ├── serializers.py
    ├── demo_run.py
    ├── sample_payload.json
    └── requirements.txt
```

6. Demo Constraints vs. Production Design

Two substitutions were necessary to complete this demo on a personal Google account without HabotConnect's Workspace admin access, and are documented here for transparency:

- The Row-Level Security grantee and BigQuery dataset owner use a personal Gmail account and the built-in projectOwners/projectReaders special groups. In production, this would be a real HabotConnect Google Workspace group such as `lsa-staff@habot.io`.
- The pipeline's deploy job prints a placeholder success message rather than deploying to a real GCP environment, since no such environment exists for this exercise.

7. Closing Note

Every control in this submission — encryption, least-privilege IAM, row-level security, automated linting, secret scanning, fail-closed quarantine logic, and DCYN validation — is enforced in code rather than relying on a person to remember a checklist. This reflects the "no reliance on hope or human memory" principle stated in the project brief.